

SIRALAMA ALGORİTMALARI

Çalışma Kağıdı | Insertion Sort · Bubble Sort · Quick Sort Kod +
İterasyon + Θ İspatı

Genel Strateji: Her algoritma için (1) kodu yaz, (2) itörasyon sayısını $\sum$ ile hesapla, (3) $c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n)$ göstererek Θ sınıfını ispat et.

1. Insertion Sort

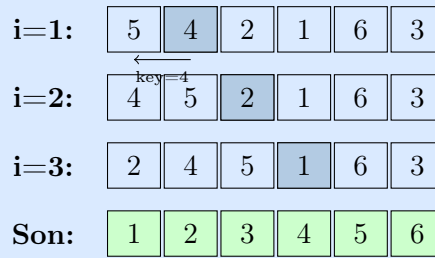
Her adımda yeni bir elemanı sol taraftaki sıralı diziye doğru konuma ekler. Kart oyununda elle sıralama yapma gibi düşünülür.

Insertion Sort

Kod

```
void Insertion_Sort(int arr[], int n) {
    int i, j, key;
    for (i = 1; i < n; i++) {
        key = arr[i];
        for (j = i-1; j >= 0 && arr[j] > key; j--)
            arr[j+1] = arr[j];
        arr[j+1] = key;
    }
}
```

Adım Adım Örnek: {5, 4, 2, 1, 6, 3}



İterasyon Analizi (En Kötü Durum)

Dış döngü: $1 + 1 \leq i < n \Rightarrow i < n - 1 \Rightarrow i, 0$ 'dan $n - 2$ 'ye gider.

İç döngü: $j = i - 1$ 'den 0'a kadar, yani en fazla i kez çalışır.

$$\sum_{i=0}^{n-2} \sum_{j=0}^{i-1} 1 = \sum_{i=0}^{n-2} i = 0+1+2+\dots+(n-2) = \frac{(n-2)(n-1)}{2} = \frac{n^2 - 3n + 2}{2}$$

$$f(n) = n^2 \text{ seçimi: } T(n) = \frac{n^2 - 3n + 2}{2}$$

$$c_1 n^2 \leq \frac{n^2 - 3n + 2}{2} \leq c_2 n^2 \implies c_1 \leq \frac{1}{2} - \frac{3}{2n} + \frac{1}{n^2} \leq c_2$$

$$n_0 = 3, c_1 = c_2 = \frac{1}{9} \text{ seçilirse koşul sağlanır:}$$

$$T(n) = \Theta(n^2)$$

En iyi durum: Dizi zaten sıralı ise iç döngü hiç çalışmaz $\Rightarrow T(n) = \Theta(n)$

Pratik Sorular – Insertion Sort

(a) {3, 1, 4, 1, 5, 9, 2, 6} dizisini Insertion Sort ile adım adım sırala. Her adımda dizinin durumunu yazın. _____

(b) En iyi durum için ($T(n) = \Theta(n)$) hangi giriş gereklidir? İç döngü neden hiç çalışmaz? _____

(c) $n = 5$ için en kötü durum kaç karşılaştırma yapar? Formülle hesaplayın. _____

2. Bubble Sort

Yan yana elemanları karşılaştırarak büyük elemanları kabarcık gibi sona taşır. Her geçişte en büyük eleman kendi yerine oturur.

Bubble Sort

Kod (Optimize – Erken Dur)

```
void bubble_sort(int arr[], int n) {
    int temp;
    bool swapped;
    for (int i = 0; i < n-1; i++) {
        swapped = false;
```

```

for (int j = 0; j < n-i-1; j++) {
    if (arr[j] > arr[j+1]) {
        temp = arr[j];
        arr[j] = arr[j+1];
        arr[j+1] = temp;
        swapped = true;
    }
}
if (!swapped) break;
}

```

İterasyon Analizi (En Kötü Durum)

Dış döngü: $0 + 1 \leq i < n - 1 \Rightarrow i < n - 2$ ($n - 1$ kez)

İç döngü: $0 + 1 \leq j < n - i - 1 \Rightarrow j < n - i - 2$, yani $n - i - 1$ kez

$$\sum_{i=0}^{n-2} \sum_{j=0}^{n-i-2} 1 = \sum_{i=0}^{n-2} (n-i-1) = (n-1) + (n-2) + \dots + 1 = \frac{(n-1)n}{2} = \frac{n^2 - n}{2}$$

$f(n) = n^2$ seçimi:

$$c_1 n^2 \leq \frac{n^2 - n}{2} \leq c_2 n^2 \implies c_1 \leq \frac{1}{2} - \frac{1}{2n} \leq c_2$$

$n_0 = 2$, $c_1 = c_2 = \frac{1}{4}$ seçilirse koşul sağlanır:

$$T(n) = \Theta(n^2)$$

En iyi durum: swapped hiç true olmaz $\Rightarrow$ tek geçiş yeterli $\Rightarrow T(n) = \Theta(n)$

Kabarcık Hareketi Görseli

Baş:	5	3	8	2	9
Pas	13	5	2	8	9
Pas	23	2	5	8	9
Son	2	3	5	8	9

Pratik Sorular – Bubble Sort

(a) {64, 34, 25, 12, 22, 11, 90} dizisi için her geçiş sonrası diziyi yazın. Kaç geçiş gerektiği? _____

(b) Insertion Sort ve Bubble Sort her ikisi de $\Theta(n^2)$. Hangisi pratikte daha hızlıdır ve neden? (Karşılaştırma vs. takas sayısını düşünün) _____

(c) swapped optimizasyonu olmadan en iyi durum ne olurdu? _____

3. Quick Sort

Böl ve fethet yaklaşımı: pivot eleman seçerek diziyi iki parçaya ayırır, sol taraf pivottan küçük, sağ taraf büyük olur; her parça yinelemeli sıralanır.

Quick Sort

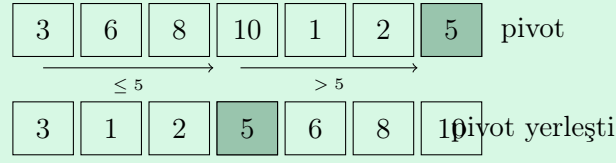
Kod

```
void qsort(int arr[], int n) {
    dq(arr, 0, n-1, n);
}

void dq(int arr[], int low, int high, int n) {
    if (low < high) {
        int pivot = partition(arr, low, high, n);
        dq(arr, low, pivot-1, n);
        dq(arr, pivot+1, high, n);
    }
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];    // son eleman pivot
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i+1], arr[high]);
    return i+1;
}
```

Partition Görseli



En Kötü Durum Analizi

En kötü durum: Her pivot seçiminde en büyük veya en küçük eleman pivot olur (zaten sıralı dizi). Bölünme dengesiz olur: $n - 1$ ve 0 boyutlu iki parça.

Yineleme bağıntısı:

$$T(n) = T(n - 1) + c \cdot n, \quad T(1) = c$$

$$\begin{aligned} T(n) &= T(n - 1) + cn \\ T(n - 1) &= T(n - 2) + c(n - 1) \\ T(n - 2) &= T(n - 3) + c(n - 2) \\ &\vdots \\ T(2) &= T(1) + c \cdot 2 \end{aligned}$$

Hepsini toplarsak $T(1) = c$ ile:

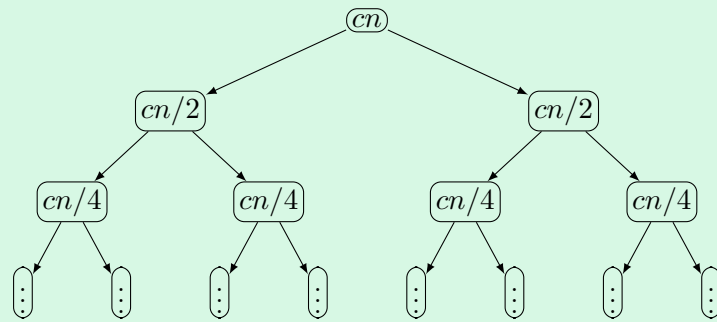
$$T(n) = c + c \cdot 2 + \dots + c \cdot n = c(1 + 2 + \dots + n) = c \cdot \frac{n(n + 1)}{2} = \boxed{\Theta(n^2)}$$

Ortalama / En İyi Durum

Pivot her seferinde diziyi tam orta bölerse: $T(n) = 2T(n/2) + cn \Rightarrow$ Master Teorem Durum 2 $\Rightarrow$

$$\boxed{T(n) = \Theta(n \log n)}$$

Yineleme Ağacı(En İyi Durum)



Her seviyede toplam: cn | Derinlik: $\log n$ | Toplam: $cn \log n$

Pratik Sorular – Quick Sort

(a) $\{10, 7, 8, 9, 1, 5\}$ dizisinde son eleman pivot olarak partition uygulaya. Partition sonrası diziyi ve pivot indeksini bulun. _____

(b) Hangi giriş Quick Sort'u en kötü duruma sokar? Neden $\{1, 2, 3, 4, 5\}$ kötü bir giristi? _____

(c) Random pivot seçimi neden beklenen karmasikliği $O(n \log n)$ 'e düşürür? _____

4. Özet ve Karşılaştırma

Algoritma	En İyi	Ortalama	En Kötü	Alan	Kararlı?	Yerinde?
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Evet	Evet
Bubble Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Evet	Evet
Quick Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$O(\log n)$	Hayır	Evet
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n)$	Evet	Hayır
Heap Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(1)$	Hayır	Evet

Önemli Notlar:

- **Kararlı (Stable):** Eşit elemanların görelî sırası korunur.
- **Yerinde (In-place):** Ek dizi gerekmez, $O(1)$ veya $O(\log n)$ ek alan yeterli.
- Quick Sort pratikte genellikle Merge Sort'tan hızlıdır (cache-friendly).
- Küçük diziler için Insertion Sort çoğu zaman en hızlısıdır.

Pratik Sorular – Karma Sorular

(a) $n = 1000$ eleman için Insertion Sort ile Quick Sort (ortalama) kaç işlem yapar? Yaklaşık değerleri karşılaştırın. _____

(b) Çoğunluğu zaten sıralı olan bir diziyi sıralamak için hangi algoritmayı seçerdiniz? Neden? _____

(c) Quick Sort'un beklenen zaman karmaşıklığı $T(n) = 2T(n/2) + cn$. Bu bağıntıyı Master Teorem ile çözün. _____

(d) Bir fonksiyon hem Insertion Sort hem Bubble Sort'u çağırıyor. Toplam zaman karmaşıklığı nedir? _____